DefineInfoTech
We Define Intellect

PROJECT REPORT

CRM Lead Management Automation

Item	Details
Project ID	DIT-P06
Project Title	CRM Lead Management Automation
Team	T06
Team Members	Madhumitha, Kanmani
Technology	Python 3, pandas, openpyxl, unittest
Submitted To	DefineInfoTech Pvt. Ltd. (DIT)
Date	November 2026

1. Introduction

1.1 Business Problem

Sales teams receive leads from many channels such as referrals, webinars, trade shows and websites. These leads arrive with inconsistent or incorrect data, duplicate entries and no clear priority. As a result, high-value leads may be contacted late, work is divided unevenly among sales representatives, and managers have no quick view of lead quality or response delays.

1.2 Objective

The objective of this project is to automate the lead-handling cycle with a Python program that cleans incoming lead data, removes duplicates, scores and grades each lead, assigns it to a sales representative, checks response deadlines, and produces reports for management.

1.3 Expected Business Outcome

- Higher data quality: invalid records are rejected with a clear reason.
- Faster response to high-value leads through Hot / Warm / Cold prioritisation.
- Fair and region-based distribution of leads among sales representatives.
- Visibility of late follow-ups (SLA breaches) and of which lead sources perform best.

1.4 Scope

The project processes a CSV file of leads and produces CSV and Excel outputs. It uses synthetic data only. No real customer or confidential data was used, in line with the DIT data privacy requirements.

2. Requirements

2.1 Functional Requirements

ID	Requirement
FR1	Read leads from a CSV file and validate mandatory fields, email, phone, dates and numeric values
FR2	Reject invalid rows with a reason, without stopping the program
FR3	Detect and remove duplicate leads (same email), keeping the earliest record
FR4	Score every lead from 0 to 100 and grade it Hot, Warm or Cold
FR5	Assign open leads to sales representatives by region (round-robin; Hot leads to Senior reps)
FR6	Calculate the follow-up due date and flag SLA breaches
FR7	Produce CSV outputs and an Excel summary report
FR8	Log every run and provide a command-line interface with input, output and reference-date options

2.2 Non-Functional Requirements

- Reliability: input validation and exception handling so bad data never crashes the program.
- Maintainability: all business rules are stored in one configuration file (config.py).

- Traceability: a log file records each run, and rejected records carry a reason.
- Testability: automated unit and end-to-end tests.

3. Workflow and Architecture

3.1 Processing Workflow

1. Read the raw leads file (data/leads_raw.csv).
2. Validate and clean each record. Invalid records go to rejected_leads.csv.
3. Remove duplicate leads. Duplicates go to duplicate_leads.csv.
4. Score and grade each unique lead.
5. Assign each open lead to a sales representative.
6. Calculate the follow-up due date and SLA breach flag.
7. Write processed_leads.csv and the Excel summary report (lead_summary.xlsx).

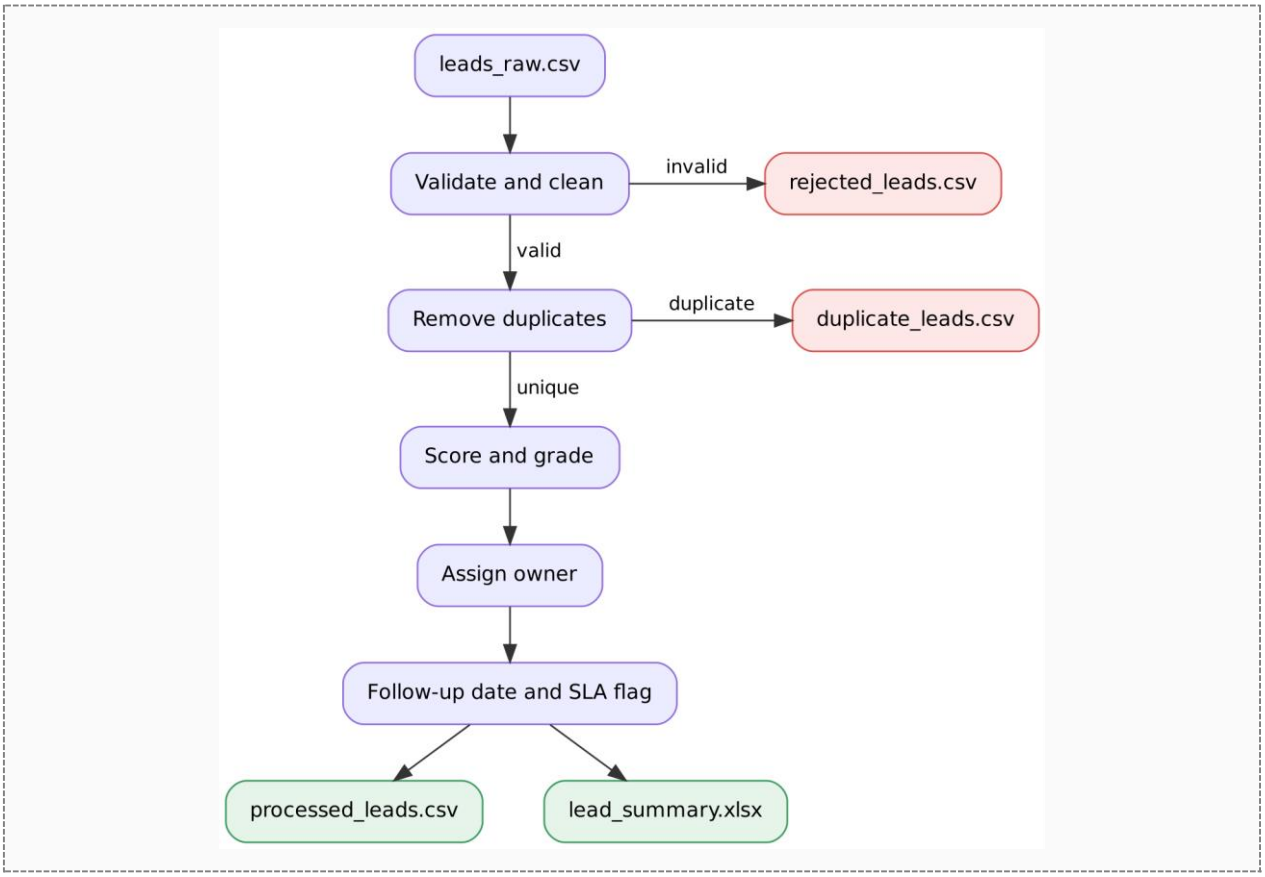


Figure 1: Project workflow

3.2 Module Structure

Module	Purpose
config.py	Central place for all business rules, thresholds and the sales team
generate_dataset.py	Creates the synthetic lead dataset, including deliberate data problems
validators.py	Validates and cleans each record; separates valid and rejected rows

Module	Purpose
dedupe.py	Finds duplicate emails and keeps the earliest lead
scoring.py	Calculates the 0–100 lead score and the Hot / Warm / Cold grade
assignment.py	Region-based round-robin assignment of open leads to representatives
followup.py	Computes follow-up due dates and SLA breach flags
reports.py	Writes CSV files and the multi-sheet Excel summary
pipeline.py / main.py	Runs all modules in order; command-line interface and logging
test_pipeline.py	Automated unit and end-to-end tests

4. Dataset and Business Rules

4.1 Input Dataset

The dataset is synthetic and was generated with a fixed random seed so that results can be repeated. It contains 126 records. Of these, 120 are normal leads and 6 are deliberate duplicates; additional records carry deliberate errors to test validation.

Field	Description
lead_id, name, company, industry	Identification and business details of the lead
email, phone	Contact details (email must be valid; phone must be a 10-digit Indian mobile number)
source	Referral, Webinar, Website, Trade Show, Social Media, Cold Call
region	North, South, East, West
created_date, status	Date the lead was created; New, Contacted, Qualified, Converted or Lost
budget, company_size	Expected budget in INR and number of employees
timeline	Purchase timeline: 0-1, 1-3, 3-6 or 6+ Months
website_visits, emails_opened	Engagement indicators

Deliberate data quality problems added for testing: 6 duplicate emails, 4 invalid emails, 3 missing phone numbers, 2 invalid dates, 2 negative budgets and 1 unknown source.

4.2 Validation Rules

- Mandatory fields: lead_id, name, email, phone, source, region, created_date.
- Email must match a valid email pattern; phone must be a valid 10-digit Indian mobile number (the +91 prefix is accepted and removed).
- Source, region, status and timeline must be from the allowed lists; text is trimmed and standardised.
- Dates must be valid; budget, company size and engagement values must be numeric and not negative.
- Missing optional values receive defaults (for example, budget 0 and status New).

4.3 Duplicate Rule

Two records with the same email address are treated as the same lead. The record with the earliest created date is kept and the others are moved to duplicate_leads.csv.

4.4 Scoring Rules (maximum 100 points)

Factor	Rule	Max
Source	Referral 25, Webinar 20, Website 15, Trade Show 15, Social Media 8, Cold Call 5	25
Budget (INR)	10,00,000+ : 25; 5,00,000+ : 20; 1,00,000+ : 10; below : 5	25
Company size	200+ : 15; 50–199 : 10; 11–49 : 6; 1–10 : 3	15
Engagement	min(15, website_visits × 2 + emails_opened)	15
Timeline	0-1 Months 20; 1-3 Months 12; 3-6 Months 6; 6+ Months 2	20

Grades: Hot = 70 or more, Warm = 40 to 69, Cold = below 40.

4.5 Assignment and SLA Rules

- Only open leads (New, Contacted, Qualified) are assigned. Converted and Lost leads are marked as closed.
- Assignment is by region using round-robin. Hot leads go to Senior representatives; Warm and Cold leads go to Junior representatives.
- First-response SLA applies to leads with status New: Hot 1 day, Warm 3 days, Cold 7 days from the created date. A lead not contacted within this time is flagged as an SLA breach.

5. Implementation

The solution is written in Python and developed module by module. Each module has a single responsibility, which makes it easy to test and modify.

5.1 Validation (validators.py)

Each row is checked against the validation rules. A valid row is converted into a clean record with standardised text, proper dates and numbers. An invalid row is stored with a reject_reason that may list several problems.

5.2 Duplicate Detection (dedupe.py)

Leads are sorted by created date, and any later record with an already-seen email is separated as a duplicate.

5.3 Scoring (scoring.py)

The score is the sum of the points for source, budget tier, company-size tier, engagement and timeline. The score is converted to a grade using the thresholds in config.py.

5.4 Assignment (assignment.py)

A counter per region and level keeps track of the next representative, so leads are distributed evenly in rotation.

5.5 Follow-up and SLA (followup.py)

The follow-up due date equals the created date plus the SLA days of the lead's grade. For New leads whose due date is before the reference date, an SLA breach flag and the number of days overdue are added.

5.6 Reporting (reports.py)

The program writes processed_leads.csv, rejected_leads.csv, duplicate_leads.csv and lead_summary.xlsx. The Excel report has the sheets All Leads, By Grade, By Source, By Owner and SLA Breaches.

5.7 Error Handling and Logging

The program reports a missing input file or missing required columns with a clear message and exit code. All steps are written to logs/pipeline.log, including every rejected record.

6. Testing

6.1 Test Approach

Automated tests were written with Python's unittest library. They check individual rules (unit tests) and the whole pipeline on the generated dataset (end-to-end test). The tests were run with the command python -m unittest -v.

6.2 Test Cases and Results

No.	Test case	Input	Expected result	Result
1	Phone cleaning	"+91 98765 43210", "12345", None	9876543210; invalid values return None	Pass
2	Invalid data rejected	Bad email, missing phone, bad date, negative budget, unknown source	Row rejected with matching reason	Pass
3	Maximum score	Referral, ₹12,00,000, 250 staff, 6 visits, 5 emails, 0-1 Months	Score 100, grade Hot	Pass
4	Minimum score	Cold Call, ₹50,000, 5 staff, no engagement, 6+ Months	Score 15, grade Cold	Pass
5	Grade boundaries	Scores 70, 69, 40, 39	Hot, Warm, Warm, Cold	Pass
6	Duplicate handling	Two leads with the same email, different dates	Earliest kept, later one removed	Pass
7	Hot to Senior; closed not assigned	Hot lead (North); lead with status Lost	Owner Arjun Rao; "N/A (closed)"	Pass

No.	Test case	Input	Expected result	Result
8	Round-robin rotation	Three Cold leads in North	Meena Iyer, Karan Singh, Meena Iyer	Pass
9	SLA breach	Hot lead created 01-Oct, checked 10-Oct; same lead with status Contacted	Breach, 8 days overdue; no breach for Contacted	Pass
10	End-to-end reconciliation	Full 126-record dataset	Valid + rejected = input; processed + duplicates = valid; 6 duplicates; 12 rejected	Pass

Summary: 10 tests were executed and all 10 passed.

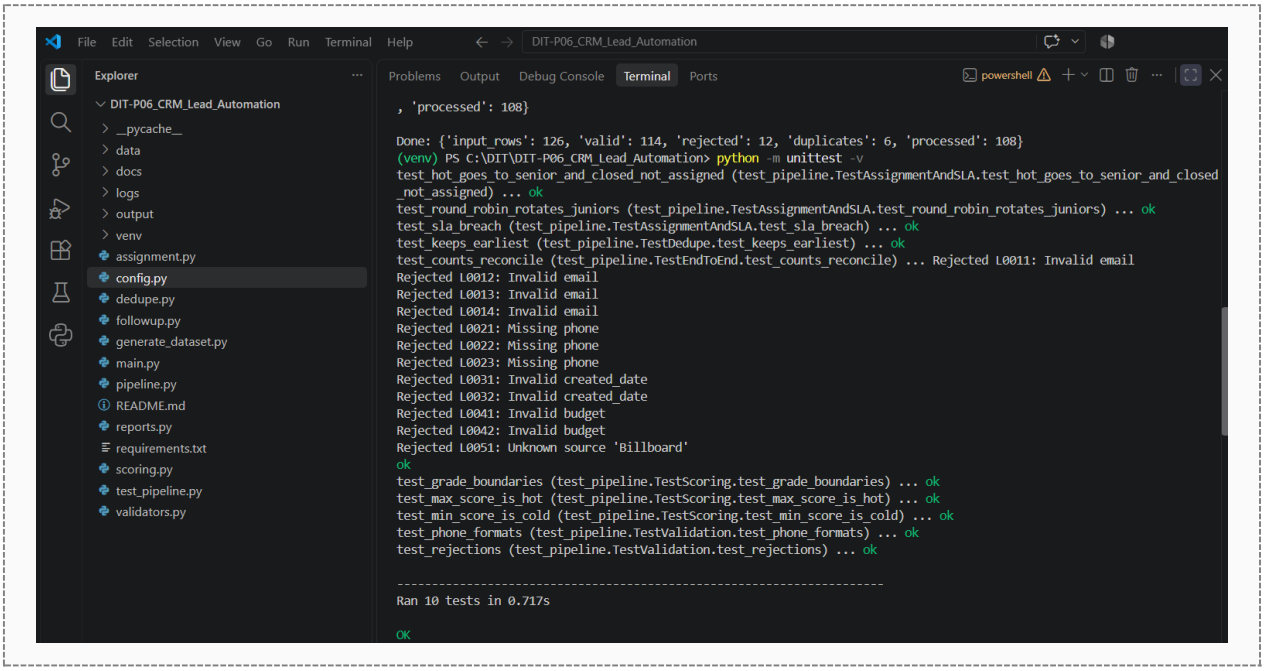


Figure 2: Unit test results

7. Results

The program was run on the 126-record dataset with 02-Nov-2026 as the reference date.

Measure	Count
Input records	126
Valid records	114
Rejected records	12
Duplicates removed	6
Processed (unique) leads	108
SLA breaches	45

Table: Pipeline summary

7.1 Leads by Grade

Grade	Leads	Average score	Total budget (INR)
Hot	39	77.7	5,40,30,000
Warm	62	57.4	3,37,50,000
Cold	7	35.3	9,20,000

7.2 Performance by Lead Source

Source	Leads	Avg score	Hot leads	Converted
Referral	19	73.1	12	0
Webinar	24	68.6	11	2
Trade Show	23	64.1	8	1
Website	16	61.4	6	0
Social Media	13	52.5	1	0
Cold Call	13	50.8	1	0

7.3 Observations

- Referral and Webinar leads have the highest average scores and produce most of the Hot leads, so these channels deserve priority.
- Cold Call and Social Media leads score lowest and give few Hot leads.
- All 45 leads with status New were past their first-response deadline at the reference date, which shows how the SLA report highlights delays. This is a result of the synthetic dates and is meant as a demonstration.
- In this sample only 7 leads are Cold. Thresholds and weights are configurable and could be tuned with real data.

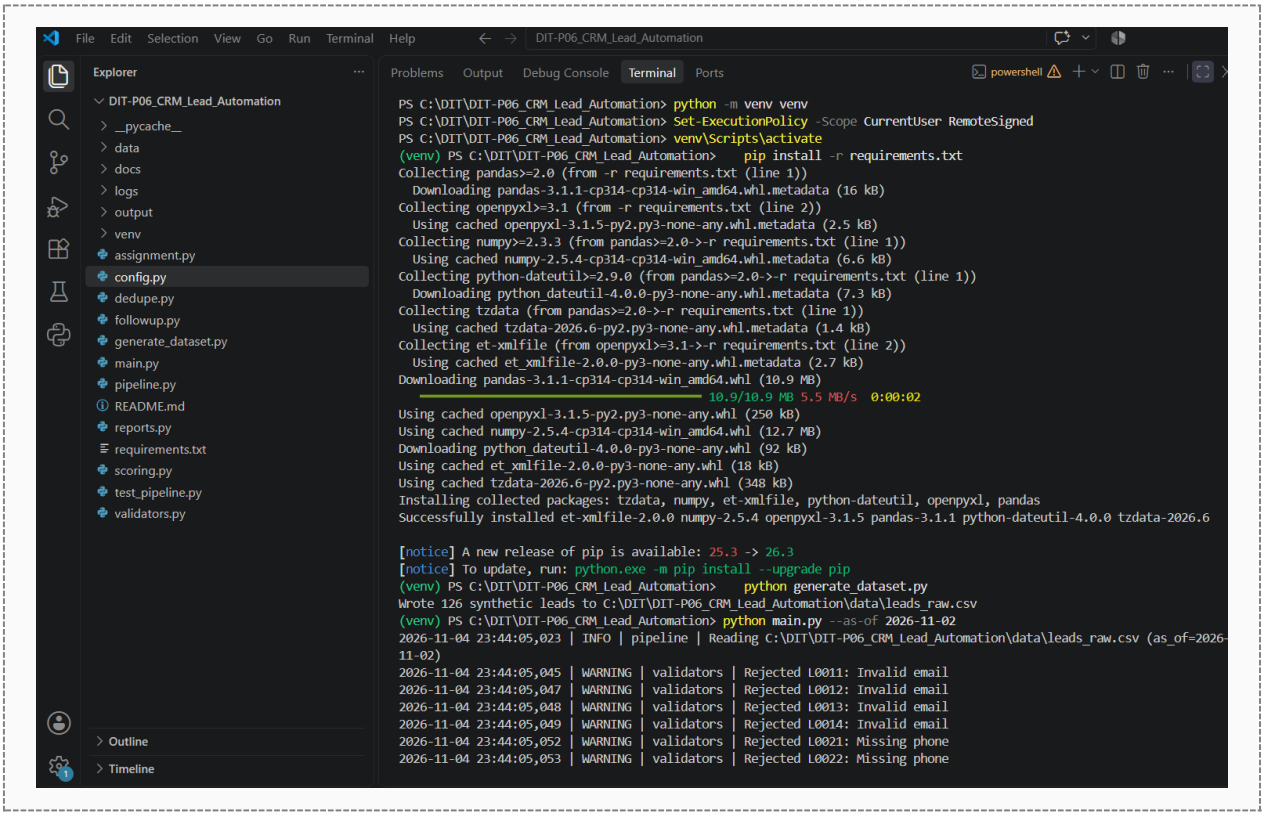
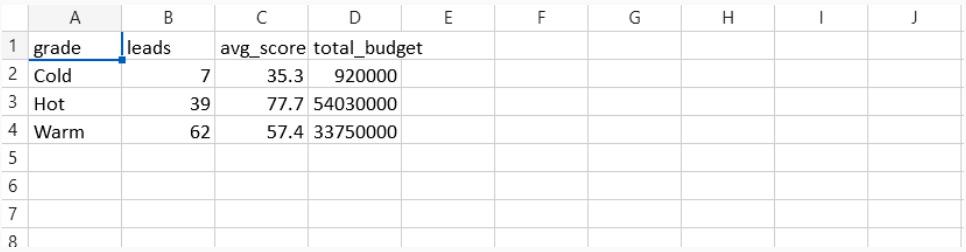


Figure 3: Program execution



	A	B	C	D	E	F	G	H	I	J
1	grade	leads	avg_score	total_budget						
2	Cold	7	35.3	920000						
3	Hot	39	77.7	54030000						
4	Warm	62	57.4	33750000						
5										
6										
7										
8										

Figure 4: Excel summary report

8. Use of AI

AI tools were used throughout the project, following the DIT AI usage guidelines. All AI-generated code and data were reviewed, run and tested by the team before use.

Stage	How AI was used	How the team verified it
Planning	Explained the business problem, proposed architecture, modules and business rules	Compared with the project requirements and the allocation workbook
Dataset	Generated a synthetic dataset with realistic fields and deliberate errors	Opened the CSV, checked fields and counted injected errors
Code	Generated Python code module by module with explanations	Read each module, ran it, and explained it within the team
Testing	Suggested test cases and expected values	Calculated scores by hand and ran all tests

Stage	How AI was used	How the team verified it
Debugging	Helped to solve environment errors (virtual environment activation, extracting the zip file)	Applied the fixes and confirmed the program ran
Documentation	Drafted README, workflow and this report	Checked every statement against the actual results

Important prompts, errors and fixes are recorded in the AI Usage Log (docs/AI_Usage_Log.md). Only synthetic data was shared with AI tools.

9. Challenges and Learnings

- Setting up the environment: PowerShell blocked the activation script, which was solved by changing the execution policy for the current user.
- Opening the project from inside a zip file placed it in a temporary folder; extracting it first solved this.
- Designing business rules that are realistic and easy to change led to a single configuration file.
- We learned that AI output must be tested: hand-calculated scores were used to verify the scoring logic.
- We learned about data validation, duplicate handling, logging, automated testing and report generation in Python.

10. Team Contribution

Area	Madhumitha	Kanmani
Requirements and workflow	Lead	Review
Dataset and business rules	Lead	Verify calculations
Validation and duplicate detection	Lead	Review and test
Scoring and assignment	Review and test	Lead
Follow-up, SLA and reports	Review and test	Lead
Testing and test results	Run and record	Add test cases
README, screenshots and demo	Screenshots	README check
Report and presentation	Sections 1–4, 5.1–5.2, 5.7	Sections 5.3–5.6, 6–10

Both team members reviewed, ran and can explain all modules of the project.

11. Conclusion and Future Enhancements

The project delivers a working and tested automation that turns a raw lead file into clean, prioritised and assigned leads with follow-up deadlines and management reports. It shows how AI can support

planning, coding, dataset preparation, testing and documentation when its output is carefully reviewed.

Future enhancements

- A live dashboard (for example with Streamlit) for managers.
- Email or messaging reminders for leads close to their SLA deadline.
- Fuzzy duplicate matching on name, company and phone.
- Reading from Excel or a CRM database instead of only CSV.
- Tuning scoring weights using the real conversion history.

Appendix A: How to Run the Project

1. Extract the project zip file to a folder (for example C:\DIT) and open the folder in VS Code.
2. Create and activate a virtual environment: python -m venv venv, then venv\Scripts\activate.
3. Install the packages: pip install -r requirements.txt.
4. Generate the dataset: python generate_dataset.py.
5. Run the automation: python main.py --as-of 2026-11-02.
6. Run the tests: python -m unittest -v.
7. Open the results in the output folder.